

COLLEGE CAPSTONE PROJECT REVIEW

YashodhaMart

E-Commerce Web Application

A full-stack, atomic transaction-driven e-commerce platform built with Next.js 14 App Router, TypeScript, Prisma ORM, and modern session security.

Presented by: Yashodha K.

Department: Artificial Intelligence and Data Science

College: JJ College of Engineering and Technology

Academic Year: 2026–2027

Abstract — Project Overview & Core Vision

Platform Introduction

YashodhaMart is a full-stack online marketplace engineered using modern web technologies: Next.js 14 App Router, TypeScript, Tailwind CSS, and Prisma ORM with a relational database backend.

Core Purpose & Mission

To solve the concurrency bottlenecks, slow page rendering, and inventory discrepancies common in traditional retail and legacy web stores by establishing deterministic stock management.

Key Features & Benefits

Provides dynamic product filtering, real-time stock-aware cart management, Indian shipping address storage, dual payment modes (COD & UPI QR), and a dedicated admin portal.

Introduction — E-Commerce Landscape & Project Motivation

The Modern E-Commerce Landscape

- Rapid Digital Transformation: High-speed internet and digital payments have made online shopping the dominant retail channel.
- Expectation of Zero Latency: Modern shoppers expect sub-second catalog transitions and instantaneous search feedback.
- High Demand for Mobile Convenience: Consumers shop on phones, requiring responsive mobile-first layouts.
- Indian Market Specifics: Need for localized address structures (PIN codes, area landmarks) and UPI payment modes.

Project Motivation & Scope

- Eliminate Retail Queue Overhead: Enable customers to order groceries and daily goods from anywhere 24/7.
- Ensure Transactional Reliability: Implement database transactions that prevent overselling of limited inventory.
- Protect Consumer Privacy: Store session credentials in HTTP-Only signed cookies rather than exposed browser storage.
- Provide Full Store Visibility: Equip shop owners with real-time analytics, inventory management, and shipment tracking.

Existing System — Limitations & Pain Points

Critical Limitations of Traditional & Legacy Systems

- ✗ Physical Shopping Constraints: Requires traveling to physical stores, fixed operating hours, and waiting in billing queues with manual invoice recording.
- ✗ Catalog Opacity & Stock Blindness: Shoppers cannot check whether an item is in stock until they physically arrive or reach the final checkout step.
- ✗ Race Conditions & Overselling: Rudimentary online systems lack atomic isolation; two shoppers can purchase the final unit simultaneously, causing order cancellation.
- ✗ Insecure Session Storage: Many legacy portals store authorization tokens in plain browser localStorage, vulnerable to Cross-Site Scripting (XSS) extraction.
- ✗ Manual Inventory Bookkeeping: Store owners update spreadsheets or physical logs manually, causing discrepancies, stockouts, and delayed shipment updates.
- ✗ Unverified Customer Reviews: Anonymous reviews in legacy stores result in spam and fake feedback, undermining customer trust.

Proposed System — Architecture & Advantages

✓ Sub-Second Full-Stack Performance

Built on Next.js 14 App Router with hybrid SSR and client hydration. Product pages render instantly with zero full-page browser refreshes.

✓ Atomic Relational Transactions

Prisma ORM transactions (`prisma.$transaction`) guarantee that order creation, stock deduction, and cart clearing execute atomically or roll back safely.

✓ Secure HTTP-Only Session Auth

Authentication uses cryptographically signed JWTs via 'jose' stored in HTTP-Only, SameSite cookies, isolating credentials from client-side scripts.

✓ Integrated UPI QR & COD Checkout

Offers both Cash on Delivery and a dynamic NPCI-compliant UPI QR Code generated in real-time with automatic order amount and reference encoding.

Project Objectives — Deliverables & Milestones

1. User Interface Usability

Create an intuitive, responsive interface using Tailwind CSS and Lucide icons that renders smoothly across desktops, tablets, and smartphones.

2. Multi-Parametric Product Discovery

Build a live catalog search with concurrent multi-filters (category, price range, star ratings, discounts, and in-stock toggles) and dynamic sort orders.

3. Stock-Aware Shopping Cart & Wishlist

Implement client-side state synchronized with server-side stock constraints, enforcing strict maxStock quantity limits and computing delivery fees.

4. Seamless Multi-Step Checkout

Develop a 3-step checkout flow incorporating Indian address management, dynamic UPI QR payment simulation, and instant order receipt generation.

5. Relational Integrity & Concurrency

Design a normalized database schema in Prisma ORM with atomic operations that increment or decrement stock safely during orders and cancellations.

6. Protected Administrative Portal

Build an administrative suite with key performance indicators, complete product catalog CRUD, shipment advancement, and user account management.

Technologies Used — Full-Stack Technical Stack

Frontend Layer

- **Next.js 14 (App Router)**
React framework with Server Components & SSR
- **React 18**
Component-based declarative UI library
- **TypeScript 5.6**
Static typing for API contracts and components
- **Tailwind CSS 3.4**
Utility-first responsive styling framework
- **Lucide React**
Lightweight, consistent SVG icon set
- **Framer Motion 11**
Declarative animations for modals and steppers

Backend & Security

- **Next.js Route Handlers**
Node.js RESTful API endpoints (/api/*)
- **Jose JWT Library**
HS256 signed tokens stored in HTTP-Only cookies
- **Bcryptjs**
Cryptographic password hashing and salting
- **QRCode & jsQR**
Dynamic UPI QR generation and image scanning
- **Validation Engine**
Server-side sanitization of inputs and payloads

Database & Tooling

- **Prisma ORM 5.21**
Type-safe database client and migration tool
- **SQLite (dev.db)**
ACID-compliant relational database engine
- **tsx Runtime**
TypeScript execution engine for seeding
- **Visual Studio Code**
Primary Integrated Development Environment
- **Git & GitHub**
Version control and collaborative repository

System Architecture — 3-Tier Layered Design

Tier 1: Presentation Layer (Client Browser)

Next.js Client Components (React 18) • Tailwind CSS • Framer Motion
Handles DOM rendering, local UI state, client-side input validation, and asynchronous fetch requests.

Tier 2: Application / API Layer (Next.js Server)

Server Route Handlers (/api/*) • JWT Authentication (jose) • Bcryptjs
Executes business logic, role-based authorization, request validation, and database orchestration.

Tier 3: Persistence Layer (Prisma ORM & SQLite)

Prisma Client Singleton (src/lib/db.ts) • SQLite Relational Engine
Enforces ACID transactions, unique constraints, foreign keys, and atomic inventory mutations.

Frontend Design — UI Hierarchy & Component System

Home Landing Page

Dynamic hero carousel, department shortcut grid, featured deals, new arrivals, and customer trust badges designed to maximize engagement.

Product Catalog & Multi-Filter

Live search input with instant debounced querying. Sidebar filters for categories, price sliders, star ratings, and discount percentages.

Product Details Page

Multi-angle image gallery, discount pricing comparison, warehouse stock limit badges, quantity spinner, wishlist toggle, and verified buyer reviews.

Multi-Step Checkout Stepper

3-step guided flow: shipping address selection/modal creation, payment selection (COD vs dynamic UPI QR), and instant order confirmation receipt.

Backend Implementation — APIs & Security Logic

Server Architecture & Security

- RESTful Route Handlers: Expressive HTTP methods (GET, POST, PUT, PATCH, DELETE) mapped under `/api/`.
- Stateless JWT Verification: Every incoming request extracts the `'ym_session'` cookie and verifies cryptographic signatures with `'jose'`.
- Role-Based Isolation (RBAC): Admin routes verify `role === 'ADMIN'` against the Admin table, preventing customer privilege escalation.
- Password Salting: Passwords hashed via `bcryptjs` with 10 salt rounds prior to persistence.

// Session Creation (`src/lib/auth.ts`)

```
const token = await new SignJWT({ ...payload })
  .setProtectedHeader({ alg: 'HS256' })
  .setIssuedAt()
  .setExpirationTime('7d')
  .sign(SECRET_KEY);

const cookieStore = cookies();
cookieStore.set('ym_session', token, {
  httpOnly: true,
  secure: process.env.NODE_ENV === 'prod',
  sameSite: 'lax',
  path: '/',
  expires: expiresAt,
});
```

Database Design — Schema & Relational Models

User & Authentication

- User (id, name, email, phone, role)
- Admin (id, name, email, role)
- Session (id, userId, token, expiresAt)

Product & Catalog

- Category (id, name, slug, parentId)
- Product (id, name, price, stock, images)
- Self-referencing category hierarchy

Cart, Wishlist & Address

- Cart (id, userId) & CartItem (quantity)
- Wishlist (id, userId) & WishlistItem
- Address (fullName, street, pincode)

Order & Verified Reviews

- Order (orderNumber, status, paymentMethod)
- OrderItem (price, quantity, snapshot)
- Review (userId, productId, rating 1-5)

CRUD Operations — Implementation in YashodhaMart

CREATE

POST Requests

- User: Registration & Session Creation
- Admin: Add New Products & Categories
- Customer: Add to Cart & Wishlist
- Checkout: Atomic Order Placement

READ

GET Requests

- Storefront: Multi-Filter Product Catalog
- Product Details: Images, MRP & Reviews
- Cart & Wishlist: Real-Time Subtotals
- Admin: KPI Metrics & Order Pipeline

UPDATE

PUT / PATCH Requests

- Admin: Product Price, Stock & Status
- Cart: Item Quantity Increment/Decrement
- Admin: Advance Order Shipment Status
- Address: Edit Saved Shipping Details

DELETE

DELETE Requests

- Admin: Delete Outdated Product Listings
- Cart: Remove Specific Cart Items
- Wishlist: Remove Bookmarked Products
- Customer: Cancel Order (Restores Stock)

Main Features & End-to-End User Workflow

1. Discovery

Browse Hero Slider, Categories, or Search by keyword with filters.

2. Item Selection

Inspect details, verify stock count, select quantity, add to cart.

3. Cart Review

Review subtotal, delivery charge breakdown (free shipping > ₹500).

4. Address Selection

Pick existing saved Indian address or create new address via modal.

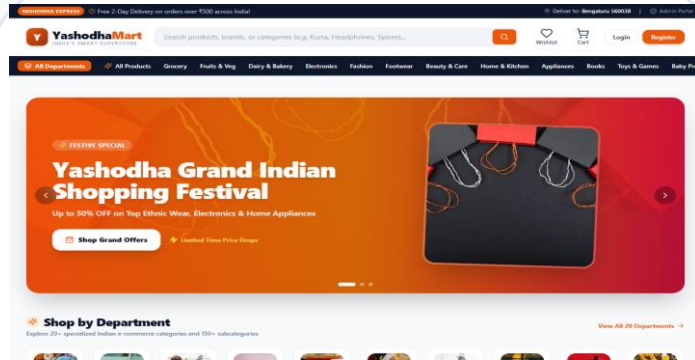
5. Payment Mode

Choose Cash on Delivery (COD) or scan dynamic UPI QR Code.

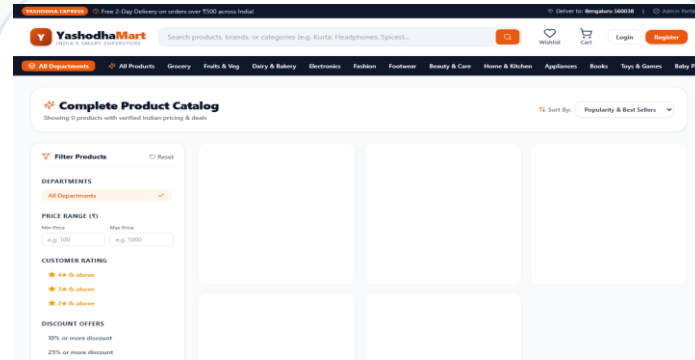
6. Order Tracking

Receive YM-ORD ID, view receipt, track status, or cancel with stock rollback.

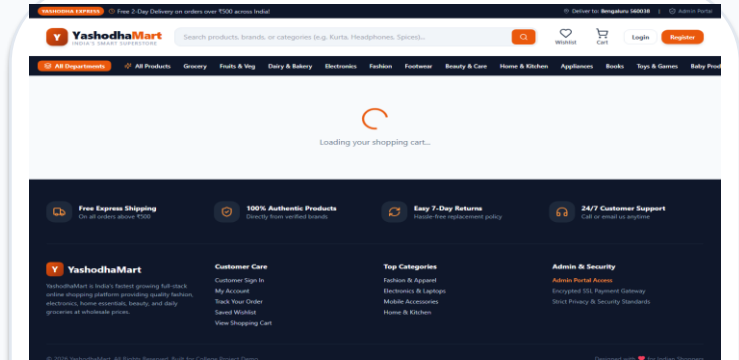
Application Screenshots — Verified Live Application



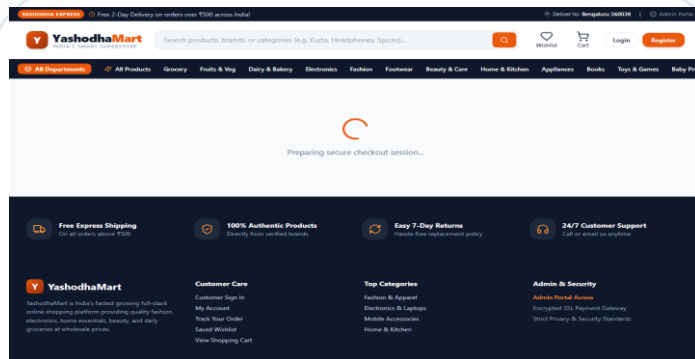
Storefront Home Page



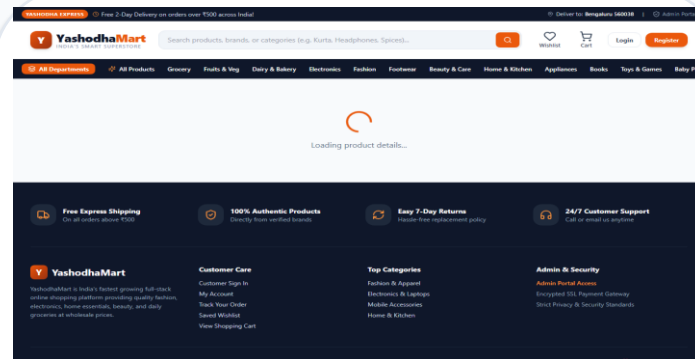
Catalog & Search



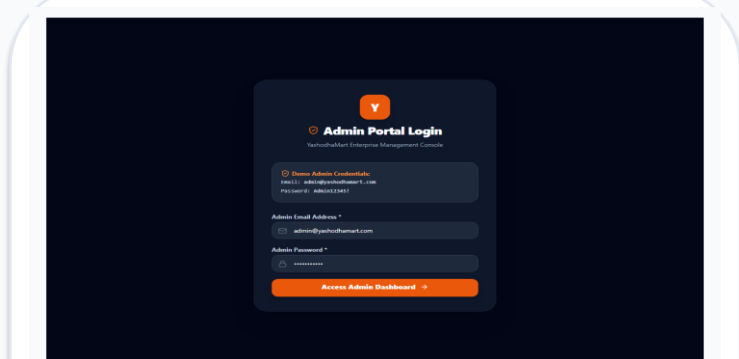
Shopping Cart



Multi-Step Checkout



Product Details



Admin Portal Login

Testing and Results — Verification & Honest Review

Authentication & Session Verification

Tested user signup, password hashing with bcrypt, JWT generation, and HTTP-Only cookie storage. Verified that invalid passwords or duplicate emails reject cleanly.

PASSED

Stock Validation & Concurrency Limits

Tested adding quantities beyond warehouse stock. The API returns HTTP 400 with an exact error message preventing over-ordering.

PASSED

Atomic Order Checkout & Decrement

Placed multi-item orders and confirmed that Order, OrderItem, stock decrement, and cart deletion complete atomically in a single transaction.

PASSED

Order Cancellation Stock Restoration

Cancelled an eligible pending order and verified that the items are incremented back into product stock counts immediately.

PASSED

Payment Processing Implementation

Cash on Delivery and dynamic UPI QR simulation function properly. Production payment gateway (Razorpay/Stripe) is planned for future release.

DEMO SIMULATION

Conclusion — Key Achievements & Takeaways

★ Full-Stack Architectural Mastery

Successfully built a complete, high-performance web platform utilizing Next.js 14 App Router, TypeScript, Tailwind CSS, and Prisma ORM.

★ Deterministic Inventory Integrity

Eliminated stock overselling and concurrency bugs through transactional database isolation (`prisma.$transaction`).

★ Enterprise-Grade Session Security

Safeguarded customer sessions through HTTP-Only cryptographically signed JWT cookies and salted bcrypt password hashing.

★ Comprehensive Dual-Surface Platform

Delivered both an intuitive customer storefront and a feature-complete administrative management console.

Future Enhancements — Technical Roadmap

1. Live Production Payment Gateway

Integrate Razorpay or Stripe webhooks with automated cryptographic HMAC-SHA256 signature verification for instant debit/credit card and NetBanking settlements.

PLANNED

2. Real-Time GPS Order Tracking

Implement Mapbox and WebSocket (Socket.io) telemetry so customers can track delivery riders in real-time on a live interactive map.

PLANNED

3. AI Recommendation Engine

Incorporate collaborative filtering and cosine similarity models to generate personalized 'Frequently Bought Together' recommendations.

PLANNED

4. Omnichannel Notification Pipeline

Integrate Twilio SMS, WhatsApp Business API, and SendGrid to send automatic dispatch and delivery alerts directly to customers' phones.

PLANNED

5. Mobile App (React Native) & PWA

Expand platform into an offline-capable Progressive Web App (PWA) and cross-platform native iOS/Android mobile apps sharing our backend APIs.

PLANNED

Thank You!

Questions & Answers (Viva Voce)

Presented by: Yashodha K.

Department of Artificial Intelligence and Data Science

JJ College of Engineering and Technology (2026–2027)

Project: YashodhaMart — Full-Stack E-Commerce Web Application